UNIVERSITY MANAGEMENT SYSTEM

A PROJECT REPORT

SUBMITTED TO

SHAHEED ZULFIQAR ALI BHUTTO

INSTITUTE OF SCIENCE AND TECHNOLOGY

BY

ASAWER YASEEN-24108109

BAKHTAWAR MEMON-24108111

KHUBAL-24108116

NEHAL-24108162

SUPERVISED BY

SIR JAVED DAHRI

INTRODUCTION:

The University Management System is a relational database project designed to manage key academic operations of a university. It stores and organizes data about students, faculty, departments, courses, exams, and enrollments.

The system establishes complex relationships such as:
- One department has many faculty members
- One faculty teaches many courses
- Many students enroll in many courses (many-to-many)

The database consists of six tables: Department, Faculty, Student, Course, Enrolls, and Exam. Primary keys, foreign keys, and constraints ensure data integrity. The design is normalized from UNF to BCNF to eliminate redundancy.

Sample queries demonstrate how to retrieve useful information like student grades, faculty course assignments, and exam schedules. This project is implemented using MySQL and phpMyAdmin.
The following entities are identified in this system:

ENTITIES AND ATTRIBUTES

1. Department

- dept_id (Primary Key)
- dept_name
- hod (Head of Department)

2. Faculty

- faculty_id (Primary Key)
- name
- dept_id (Foreign Key → Department)
- hire_date

3. Student

- student_id (Primary Key)
- name
- email
- phone

4. Course

- course_id (Primary Key)
- title
- dept_id (Foreign Key → Department)
- faculty_id (Foreign Key → Faculty)
- credits

5. Enrolls (Relationship Table)

- student_id (Foreign Key → Student)
- course_id (Foreign Key → Course)
- enroll_date
- grade

6. Exam

- exam_id (Primary Key)
- course_id (Foreign Key → Course)
- exam_date
- max_marks

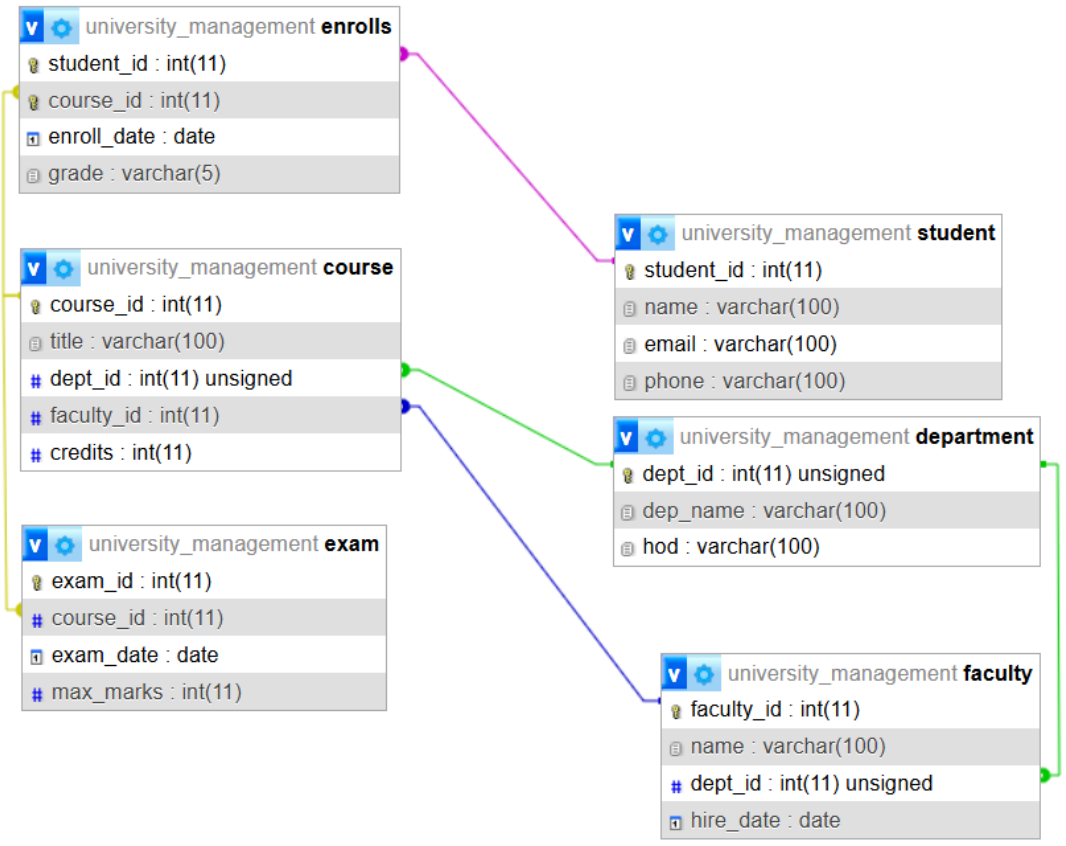
ER DIAGRAM (with proper cardinality & participation)

Entities and Relationships:

1. **Department → Faculty**
    - One department has many faculty members
    - Each faculty belongs to exactly one department
    - Cardinality: 1:N
    - Participation: Faculty (Total), Department (Partial)
2. **Department → Course**
    - One department offers many courses
    - Each course belongs to exactly one department
    - Cardinality: 1:N
    - Participation: Course (Total), Department (Partial)
3. **Faculty → Course**
    - One faculty teaches many courses
    - Each course is taught by exactly one faculty
    - Cardinality: 1:N
    - Participation: Course (Total), Faculty (Partial)
2. **Student ↔ Course (through Enrolls)**
    - Many students take many courses
    - Many courses are taken by many students
    - Cardinality: M:N
    - Participation: Both are Partial
3. **Course → Exam**
    - One course has many exams
    - Each exam belongs to exactly one course
    - Cardinality: 1:N

- Participation: Exam (Total), Course (Partial)

ER DIAGRAM:



RELATIONAL SCHEMA

Department Table

- dept_id (Primary Key)
- dept_name
- hod (Head of Department)

Faculty Table

- faculty_id (Primary Key)

- name
- dept_id (Foreign Key references Department)
- hire_date

Student Table

- student_id (Primary Key)
- name
- email
- phone

Course Table

- course_id (Primary Key)
- title
- dept_id (Foreign Key references Department)
- faculty_id (Foreign Key references Faculty)
- credits

Enrolls Table

- student_id (Foreign Key references Student)
- course_id (Foreign Key references Course)
- enroll_date
- grade
- Primary Key: (student_id, course_id) composite

Exam Table

- exam_id (Primary Key)
- course_id (Foreign Key references Course)
- exam_date
- max_marks

NORMALIZATION STEPS:
UNF (Unnormalized Form)

One big table with repeating groups:

Student_Course_Info (student_id, student_name, student_email, student_phone, {course_id, course_title, course_credits, faculty_name, grade, exam_date, max_marks})

Problems in UNF:

- Repeating groups for each course taken by a student
- Data redundancy
- Insert, update, delete anomalies
- Wasted storage space

1NF (First Normal Form)

Remove repeating groups by creating separate tables:

Table 1: Student (student_id, student_name, student_email, student_phone)

Table 2: Enrollment (student_id, course_id, grade, exam_date, max_marks)

Table 3: Course_Teacher (course_id, course_title, course_credits, faculty_name)

Achieved 1NF because:

- No repeating groups
- Each cell contains atomic (single) value
- Each record is unique

Problems Remaining in 1NF:

- Partial dependencies (exam_date and max_marks depend only on course_id, not on student_id)
- Transitive dependencies (faculty_name may depend on faculty_id)

2NF (Second Normal Form)

Remove partial dependencies. In Enrollment table, exam_date and max_marks depend only on course_id, not on the composite key (student_id, course_id).

So split Enrollment table:

Table 1: Enrollment (student_id, course_id, grade)

Table 2: Exam (course_id, exam_date, max_marks)

Now:

- Enrollment table: grade depends on both student_id and course_id
- Exam table: exam_date and max_marks depend on course_id only

Course_Teacher table remains the same.

Achieved 2NF because:

- No partial dependencies
- Every non-key attribute depends on complete primary key

Problems Remaining in 2NF:

- Transitive dependency in Course_Teacher table (faculty_name may depend on faculty_id)

3NF (Third Normal Form)

Remove transitive dependencies. In Course_Teacher table, if we add faculty_id and faculty has attributes like department, then course_title depends on course_id and faculty_name depends on faculty_id.

So split Course_Teacher table:

Table 1: Course (course_id, course_title, course_credits, faculty_id)

Table 2: Faculty (faculty_id, faculty_name, dept_id)

Now:

- Course table: all attributes depend only on course_id
- Faculty table: all attributes depend only on faculty_id

Add Department table to avoid transitive dependency from faculty to department:

Table 3: Department (dept_id, dept_name, hod)

Achieved 3NF because:

- No transitive dependencies
- Every non-key attribute depends only on the primary key

BCNF (Boyce-Codd Normal Form)

Check for overlapping candidate keys:

In our design, each table has exactly one candidate key (the primary key). There are no overlapping candidate keys.

Therefore, all tables are in BCNF because:

1. Every determinant is a candidate key
2. No functional dependency violates BCNF rules

COMPLEX RELATIONSHIPS AND CONSTRAINTS

COMPLEX RELATIONSHIPS

Relationship 1: Department to Faculty

- Type: One to Many (1:N)
- Description: One department has many faculty members
- Example: Computer Science department has Dr. Yousaf as faculty
- Constraint: Each faculty must belong to exactly one department

Relationship 2: Department to Course

- Type: One to Many (1:N)
- Description: One department offers many courses
- Example: Computer Science department offers Programming course
- Constraint: Each course must belong to exactly one department

Relationship 3: Faculty to Course

- Type: One to Many (1:N)

- Description: One faculty teaches many courses
- Example: Dr. Yousaf teaches Programming course
- Constraint: Each course must be taught by exactly one faculty

Relationship 4: Student to Course (via Enrolls)

- Type: Many to Many (M:N)
- Description: Many students take many courses
- Example: Ali Raza and Sara Khan both take Programming course
- Constraint: This is implemented using Enrolls table as a bridge

Relationship 5: Course to Exam

- Type: One to Many (1:N)
- Description: One course has many exams
- Example: Programming course has one exam
- Constraint: Each exam must belong to exactly one course

CONSTRAINTS IMPLEMENTED

Primary Key Constraints:

1. Department table: dept_id is unique identifier
2. Faculty table: faculty_id is unique identifier
3. Student table: student_id is unique identifier
4. Course table: course_id is unique identifier
5. Enrolls table: (student_id + course_id) together is unique
6. Exam table: exam_id is unique identifier

Foreign Key Constraints:

1. Faculty.dept_id must exist in Department.dept_id
2. Course.dept_id must exist in Department.dept_id
3. Course.faculty_id must exist in Faculty.faculty_id

4. Enrolls.student_id must exist in Student.student_id
5. Enrolls.course_id must exist in Course.course_id
6. Exam.course_id must exist in Course.course_id

Referential Integrity Constraints:

1. ON DELETE RESTRICT: Cannot delete a department if faculty belongs to it
2. ON DELETE RESTRICT: Cannot delete a department if courses belong to it
3. ON DELETE RESTRICT: Cannot delete a faculty if courses are assigned to them
4. ON DELETE CASCADE: If student is deleted, their enrollments are also deleted
5. ON DELETE CASCADE: If course is deleted, its enrollments and exams are deleted
6. ON DELETE CASCADE: If course is deleted, its exams are deleted
7. ON UPDATE CASCADE: If department ID changes, it updates in faculty and courses
8. ON UPDATE CASCADE: If faculty ID changes, it updates in courses
9. ON UPDATE CASCADE: If student ID changes, it updates in enrollments
10. ON UPDATE CASCADE: If course ID changes, it updates in enrollments and exams

Data Type Constraints:

1. INT UNSIGNED: All ID columns cannot have negative values
2. VARCHAR(100): Name fields have maximum length of 100 characters
3. DATE: Date fields must be in YYYY-MM-DD format
4. AUTO_INCREMENT: ID fields automatically generate next available number

5. NOT NULL: Primary key and foreign key columns cannot be empty

Business Rule Constraints:

1. A faculty cannot exist without a department
2. A course cannot exist without a department and a faculty
3. A student can enroll in multiple courses
4. A course can have multiple students enrolled
5. A course can have multiple exams
6. Grade values can be A, A-, B+, B, B-, C+, C, D, F (standard grading system)

SAMPLE TABLES:

DEPARTMENT TABLE

dept_id	dept_name	hod
1	Computer Science	Dr. Ahmed
2	Mathematics	Dr. Fatima

FACULTY TABLE

faculty_id	name	dept_id	hire_date
1	Dr. Yousaf	1	2020-08-15
2	Dr. Sana	2	2019-01-10

STUDENT TABLE

student_id	name	email	phone
1	Ali Raza	ali@uni.edu	03001234567
2	Sara Khan	sara@uni.edu	03009876543

COURSE TABLE

course_id	title	dept_id	faculty_id	credits
1	Programming	1	1	3
2	Calculus	2	2	4

ENROLLS TABLE

student_id	course_id	enroll_date	grade
1	1	2025-01-10	A
2	1	2025-01-12	B+

EXAM TABLE

exam_id	course_id	exam_date	max_marks
1	1	2025-05-15	100
2	2	2025-05-20	100

PHPMYADMIN WORK:

TABLES

	Table	Action	Rows	Type	Collation	Size	Overhead
☐	course	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_general_ci	48.0 KiB	-
☐	department	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐	enrolls	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐	exam	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐	faculty	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐	student	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_general_ci	16.0 KiB	-
	6 tables	Sum	14	InnoDB	utf8mb4_general_ci	176.0 KiB	0 B

TABLES DATA COUNT

Table_Name	Total_Rows
Department	4
Faculty	2
Student	2
Course	2
Enrolls	2
Exam	2

WHAT GRADE DID THEY GET AND THE COURSE DID THEY TAKE

Student_Name	Course_Title	Grade
Ali Raza	Programming	A
Sara Khan	Programming	B+

WHICH FACULTY TAUGHT THE COURSES

Faculty_Name	Course_Title	Department
Dr. Yousaf	Programming	Computer Science
Dr. Sana	Calculus	Artificial Intelligence

EXAM DETAILS

Course_Title	Exam_Date	Max_Marks
Programming	2025-05-15	100
Calculus	2025-05-20	100

DEPARTMENT WISE FACULTY COUNT

Department	Total_Faculty
Computer Science	1
Artificial Intelligence	1
Computer Science	0
Mathematics	0

DEPARTMENT WISE COURSE COUNT

Department	Total_Courses
Computer Science	1
Artificial Intelligence	1
Computer Science	0
Mathematics	0

STUDENT'S COURSE AND GRADES (LEFT JOIN)

Student	Course	Grade
Ali Raza	Programming	A
Sara Khan	Programming	B+

EVERY EXAM IS FOR A SPECIFIC COURSE

exam_id	Course	exam_date	max_marks
1	Programming	2025-05-15	100
2	Calculus	2025-05-20	100

STUDENT WHO GOT AN A GRADE

name	title	grade
Ali Raza	Programming	A

COMPLETE DATA OF STUDENTS

		student_id	name	email	phone
☐	Edit Copy Delete	1	Ali Raza	ali@uni.edu	03001234567
☐	Edit Copy Delete	2	Sara Khan	sara@uni.edu	03009876543